Prometheus: Towards Topology-Protected Continual Acquisition

Working manuscript aligned with the six research tasks

Prometheus Research Project

Research draft: 1 November 2026

Abstract

How can learning systems construct, compose, retain, and improve their own computational mechanisms? We investigate executable operations represented by monodromy, and distinguish learning an operation class from adapting its continuous implementation. This working manuscript records current mechanisms, preliminary continual-acquisition measurements, conventional explanations, and the experiments still required for a general architecture claim. Existing synthetic results demonstrate stage-boundary retention under a compact shared representation; a matched ordinary constrained learner reproduces those results. A separate terminal-feedback diagnostic learns a new loop class while retaining an old operation, and exposes the remaining coupling between operation formation and neural calling. No topology-specific competitive advantage or general-purpose architecture is established by the present evidence.

Scientific objective. The target is an original computational learning architecture with broad technical significance, not a favorable result on one finite task. The six sections below correspond exactly to the human-defined research tasks. Manuscript completion does not establish that the associated scientific acceptance criteria have passed. The current method is maintained in `water/method.md`; this document develops its paper exposition from preserved, versioned evidence.

Contents

1 Continual acquisition and capability growth **1**
- 1.1 Representations, operations, and calling . . . 1
- 1.2 Learning operations from terminal continuation profiles . . . 2
- 1.3 Binding protection and neural–native consistency . . . 3
- 1.4 Six-stage acquisition measurements . . . 3

2 Learning executable monodromy classes **5**
- 2.1 Covering space, marked states, and executable composition . . . 5
- 2.2 What safe implementation deformation does and does not learn . . . 5
- 2.3 Terminal boundary feedback and neural credit . . . 6
- 2.4 A sufficient protected transaction . . . 7
- 2.5 Executed class acquisition and the unresolved advantage . . . 7

3 A trainable loop-based Prometheus Block **8**
- 3.1 Neural state and computation . . . 8
- 3.2 One learning transaction, three retention obligations . . . 9
- 3.3 Composition and a constructive module interface . . . 10
- 3.4 Executed hybrid V1: operation formation without task acquisition . . . 10

4 TODO 4: Transfer to Real-World Continual Learning **11**
4.1 Fixed audio protocol . . . 11
4.2 Fixed visual protocol . . . 11
4.3 Memory, initialization, and cost . . . 11
4.4 Evidence boundary . . . 12

5 Benchmarking and competitive evaluation **12**
5.1 Five-layer evaluation scope . . . 12
5.2 Information, development budgets, and final evaluation . . . 13
5.3 Fixed real-data protocols and strong competitors . . . 13
5.4 Measured baseline results at the historical B1 budget . . . 14
5.5 Costs and causal attribution . . . 14
5.6 Statistical reporting and completion status . . . 15

6 Theory, evidence boundaries, and reproducibility **16**
6.1 Continuation witnesses and finite-state identification . . . 16
6.2 Binding-level protection under state expansion . . . 16
6.3 What continuous-loop protection establishes . . . 17
6.4 A sufficient soft/native consistency bound . . . 17
6.5 Executed evidence and version lineage . . . 18
6.6 Independent checking and limits of present claims . . . 18

1 Continual acquisition and capability growth

Scope of the implemented system. Our present acquisition system learns finite recurrent computations from terminal examples, installs the inferred operations in neural parameters, and subsequently trains and maintains a differentiable calling interface. This is a hybrid identification–installation–gradient-learning procedure. Its successful synthetic executions establish a working accumulation mechanism within the tested task family; they do not yet establish that topology improves acquisition or parameter growth. In particular, an ordinary constrained implementation with the same formation and maintenance rules matches the measured results.

1.1 Representations, operations, and calling

A task example consists of an observable initial state $x \in \{1, \dots, N\}$, a token word $w = (a_1, \dots, a_L)$, a public context c, and a terminal label $y \in \{1, \dots, N\}$. The public context is necessary in these synthetic streams because identical tokens have different meanings in different tasks. It is not a hidden-state label. The learner does not receive target transition tables, intermediate execution states, or correct call routes.

The shared neural state is a row distribution $\alpha \in \mathbb{R}^{1\times D}$. Let $E \in \mathbb{R}^{N\times D}$ and $R \in \mathbb{R}^{D\times N}$ denote encoder and readout logits. At temperature $\tau > 0$, encoding and decoding use row softmaxes, denoted $\overline{E}$ and $\overline{R}$. A bank contains K learned operations and a provided identity operation $P_0 = I_D$. The latter is an execution substrate primitive, not a learned discovery. For token a in context c, a

learned network produces S slot distributions $q_{c,a,s}(k)$, $k \in \{0, \dots, K\}$. Soft execution is

$$\alpha_0 = \overline{E}_{x,:}, \tag{1}$$

$$T_{c,a} = \prod_{s=1}^{S} \left(\sum_{k=0}^{K} q_{c,a,s}(k)\, \overline{P}_k \right), \tag{2}$$

$$\alpha_\ell = \alpha_{\ell-1} T_{c,a_\ell}, \qquad \widehat{p}(y \mid x, w, c) = (\alpha_L \overline{R})_y. \tag{3}$$

Products are ordered left to right in execution order because states are row vectors. Hard execution selects the encoder state, slot calls, operation matchings, and decoded label. For a destination permutation π_k, $(P_k)_{ij} = \mathbf{1}\{j = \pi_k(i)\}$; consequently $e_i P_k = e_{\pi_k(i)}$. The continuous monodromy realization executes the selected operation through path lifting. Its mathematical role and current coupling to learning are developed in the monodromy section.

The compact bank stores destination coordinates $\xi \in \mathbb{R}^{K \times D}$ and generates matching scores

$$W_{kij} = -8(j - \xi_{ki})^2. \tag{4}$$

Every permutation is representable by setting $\xi_{ki} = \pi_k(i)$. The persistent bank therefore contains KD learned scalars, although forming and differentiating its dense scores still incurs KD^2 temporary work and storage. The ordinary constrained control receives this identical compression; it is not a topology-specific saving.

With token vocabulary size $|\mathcal{A}|$, C allocated contexts, embedding width H, and S slots, the implemented shared model has

$$P_t = 2ND_t + (|\mathcal{A}| + 1 + C)H + S(K_t + 1)(H + 1) + K_t D_t + 4. \tag{5}$$

The final four scalars parameterize the shared geometric deformation. They are absent from the ordinary constrained control. Preallocated context embeddings and every frozen parameter are included. The reported runs use $|\mathcal{A}| = 6$, $C = 6$, $H = 48$, and $S = 2$.

1.2 Learning operations from terminal continuation profiles

For a fixed context, write $F_c(x, w)$ for observed terminal feedback. A history $h = (x, p)$ has a finite continuation profile

$$\sigma_{\mathcal{S}}(h) = \big(F_c(x, ps)\big)_{s \in \mathcal{S}}. \tag{6}$$

The implemented observation table uses the empty word and all one-token words as both prefixes and suffixes. Its successor check also consults words of length three. All such observations come from the supplied TRAIN examples. Different profiles provide explicit state-separation witnesses; equal profiles are a provisional finite-data equivalence, not a claim of equivalence under every future continuation.

The formation routine partitions observed histories by their profiles, checks that observed successors close on the resulting classes, and rejects conflicting or nonreversible successors. If the number of separated classes exceeds D, the workspace grows. An assignment then aligns these classes to the current encoder/readout coordinates. Already learned operations are reused when their actions agree on every constrained reachable state. Otherwise, a constrained matching completes an executable permutation, which is checked against the entire current TRAIN set before neural installation. Candidate search, matching, full-data checks, state expansion, and installation are all part of learning cost. This procedure does not learn from hidden teacher states, but does exploit the deliberately extensive short-word coverage of this benchmark.

After installation, the learner executes real gradient updates of the neural encoder, readout, coordinates, embeddings, and calling network. In the measured streams, new-task validation already equals one immediately after formation and installation, before these updates. Thus the first acquisition in these runs is attributable to terminal-profile identification and installation, rather than to the subsequent 600 Adam updates alone.

1.3 Binding protection and neural–native consistency

Protecting an operation permutation alone does not protect a task: changing the input encoding or call route can destroy an old computation even if every bank operation remains unchanged. The implementation therefore stores admitted native computation references and checks a transition-closed relation between their reachable states and the updated computation. Shared proposals are accepted, backtracked, or rejected according to this binding-level contract. The relation permits state expansion and alternative internal coordinates when they preserve decoded execution. It is a conventional semantic protection condition, distinct from continuous-loop homotopy protection.

The soft interface requires an additional maintenance step. At each task boundary, the implemented consistency refresh sharpens the model's own current encoder, decoder, operation classes, and both original call slots. It obtains no new task labels and adds no parameters. It preserves the current native graph while increasing the probability assigned to its execution path. A sufficient path-mass condition is stated in Section 6. Frozen models preserve aliased frozen endpoints during this refresh. The same maintenance is available to ordinary constrained and modular controls.

1.4 Six-stage acquisition measurements

Each stream contains six tasks. The first three use unknown reversible N-state operations. The next three require a hidden-memory workspace with $2N$ recurrent states and only N observable output labels. The target hidden mode and transition rules are never provided to the learner. TRAIN contains all initial states paired with every word of length at most three, followed by random words of lengths four through eight: 8,192 examples per task for $N = 8, 16$ and 16,384 for $N = 32$. The larger pool is required to include the $259N = 8,288$ mandatory short examples at $N = 32$.

Validation, IID, OOD, and length-16 evaluations each contain 512 examples per task. IID and validation use lengths at most eight and can overlap the exhaustive short training coverage; reported OOD uses lengths nine through twelve. Length-16 examples are disjoint from TRAIN by length. These are finite held-out computations, not evidence about arbitrary lengths. The present prototype is bounded to lengths at most 18 and does not supply the required length-32/64 publication measurements.

Define $c_i(t) = 1$ when task i meets both neural and deployed thresholds at the end of stage t: IID accuracy at least 0.95 and both OOD and length-16 accuracy at least 0.90. Admission additionally requires validation accuracy at least 0.95. The strictly retained boundary count is

$$Q_t = \sum_{i \le t} \prod_{u=i}^{t} c_i(u). \tag{7}$$

We report $\Delta P_t / \Delta Q_t$ only when a positive new stable capability is obtained without losing an already stable one. The first acquisition pays the entire initial allocation, $P_{-1} = 0$. This is a stage-boundary criterion, not an unobserved guarantee between measurements.

At $N = 8$, seed 0, the final bank contains 38 learned operations and $D = 16$; the analogous final workspaces are $D = 32$ and 64 at the larger scales. The $N = 32$ runs use a factored realization

Scale	P_0	P_1	P_2	P_3	P_4	P_5	Completed seeds
$N = 8$	1596	2232	2868	3946	4630	5314	0, 1, 2
$N = 16$	2036	2720	3404	5130	5910	6690	0, 1, 2
$N = 32$	3684	4464	5244	9034	10006	10978	0

Table 1: Exploratory, boundary-maintained topology streams. For every completed seed shown, $Q_t = 1, 2, 3, 4, 5, 6$ and every seen-task neural/deployed accuracy equals one on each of the four measured splits at each recorded stage boundary. These are the results available at this manuscript checkpoint, not completion of the five-final-seed publication protocol.

backend that preserves the learned finite operation semantics without extra trainable parameters. This engineering change and the differing acquisition pools prevent interpreting the table as a controlled timing-scaling law.

Boundary maintenance resolves a measured neural/native mismatch, rather than proving that the neural interface never forgets. In the repaired $N = 8$, seed-0 stream, the minimum recorded old-task soft accuracies in the successive later stages are approximately $0.348, 0.309, 0.252, 0.242, 0.178$, before returning to one at the maintained boundary. Recorded deployed execution remains correct. The original unrefreshed version retained only one task under the strict boundary-stability criterion; its failed trajectory is preserved separately.

The strongest matched finite-program control is decisive for attribution: ordinary constrained compact learning attains the same six-task boundary result at $N = 8$ and $N = 16$, seed 0, with respectively 5,310 and 6,686 parameters, exactly four fewer than the topology model. A repaired frozen-backbone policy and separate task modules also attain six retained tasks at $N = 8$, with 6,384 and 8,664 parameters. Therefore these measurements support compact shared acquisition and a useful consistency repair, but do not isolate a monodromy advantage in parameter growth. Strong neural competition, additional scales and seeds, and topology-specific causal learning effects remain separate evidence requirements.

Evidence locations. The frozen seed-0 growth and learning-curve extract is `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/parameter-boundary-growth-evidence.json` and `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/parameter-boundary-learning-curves.csv`. This extract predates the additional seeds. Their original summaries, raw updates, checkpoints, and synchronization hashes are under `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/runs/`, including `parameter-n8-seed1-topology-full-consistent`, `parameter-n8-seed2-topology-full-consistent`, `parameter-n16-seed1-topology-full-consistent`, and `parameter-n16-seed2-topology-full-consistent`. The $N = 32$, seed-0 lineage is preserved in the `parameter-n32-seed0-topology-through0-consistent`, `parameter-n32-seed0-topology-resumed-through2-consistent`, `parameter-n32-seed0-topology-resumed-through3-consistent`, and `parameter-n32-seed0-topology-resumed-consistent` run directories.

2 Learning executable monodromy classes

The mechanism studied here separates two operations that are easily conflated: changing the implementation of an existing computation, and acquiring a different executable computation. A deformation that is certified to preserve a based-loop homotopy class can support the former, but cannot by itself accomplish the latter. We therefore develop a learning interface with class-changing

proposals for unadmitted operations and class-preserving transactions for admitted operations. The current evidence establishes this interface in a small supplied covering; it does not yet establish an independent topological advantage over equally informed conventional optimization.

2.1 Covering space, marked states, and executable composition

For an integer $D \geq 2$, consider the supplied polynomial family

$$F_D(z,u) = z^D - Dz - u, \qquad \mathcal{C}_D = \{-(D-1)\zeta : \zeta^{D-1} = 1\}, \qquad U_D = \mathbb{C} \setminus \mathcal{C}_D. \tag{8}$$

Projection onto u defines an unbranched D-sheet covering over U_D. We fix the control basepoint $u_* = 0$ and mark its D distinct roots once. Lifting a based loop $\gamma : [0,1] \to U_D$ transports each marked root to another root in the same fiber. Its endpoint action is a permutation $\Pi_\gamma = \rho_D([\gamma])$. For chronological concatenation, defined so that $\gamma_1 \star \gamma_2$ executes γ_1 first,

$$\Pi_{\gamma_1 \star \gamma_2} = \Pi_{\gamma_2} \circ \Pi_{\gamma_1}. \tag{9}$$

An internal state is a sheet index, not a freshly initialized root at every call. Equation (9) consequently specifies an executable, generally noncommutative composition rule.

The executed class-acquisition diagnostic uses $D = 3$. The roots at $u_* = 0$ are labelled $0, 1, 2$ in the order $(-\sqrt{3}, 0, \sqrt{3})$, and the critical controls are -2 and 2. Its one-parameter family is the polygonal loop

$$\gamma_a : 0 \longrightarrow a + i \longrightarrow a - i \longrightarrow 0, \qquad \Pi_{\gamma_a} = \begin{cases} B = (12), & a < -2, \\ I, & -2 < a < 2, \\ A = (01), & a > 2. \end{cases} \tag{10}$$

The singular cases $a = \pm 2$ are not executable. For example, $\Pi_B(\Pi_A(0)) = 2$, whereas $\Pi_A(\Pi_B(0)) = 1$; the order of calls matters. The permutation labels in (10) describe the independently checked outcome, rather than an operation table provided to the learner. The numerical executor actually continues all three roots with a safeguarded predictor/Newton corrector. Its full-fiber endpoint observations, known covering, critical controls, and numerical costs are available to every comparison arm. Thus the protocol combines terminal task labels with full-fiber execution observations; it is not a single-sheet-only observation protocol. The executed source and independent reconstruction are preserved in `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/versions/code-parameter-loop-boundary-learning-v1/loop_boundary_learning.py` and `.deep-work/2026-10-31T234403-0400-v2/supervision/reviews/review-038-loop-boundary-learning.md`.

2.2 What safe implementation deformation does and does not learn

An earlier construction first identifies a permutation, compiles it into a loop of distinct root configurations, and then changes its geometry using a time-dependent ambient shear. Writing a point as $x + iy$, this shear is

$$\Phi_{\omega,t}(x + iy) = x + i\left[y + \sin^2(\pi t/T) h_\omega(x)\right], \qquad h_\omega(x) = \sum_\ell \omega_\ell \tanh(v_\ell x + b_\ell), \tag{11}$$

where v_ℓ, b_ℓ are fixed. Every finite ω gives a global diffeomorphism, with an explicit inverse obtained by subtracting the same vertical displacement. At both execution endpoints it is the identity.

Interpolating ω therefore gives a collision-free homotopy of *configuration-space* loops with a fixed base configuration, preserving their braid and endpoint permutation. This is a useful within-class learning freedom, but it is not a mechanism for discovering a new permutation. It is also a different construction from the control-plane covering in (8); the two spaces must not be identified without an additional map between them.

The new class-learning interface instead trains the raw vertices of a loop in the fixed control plane. A controller $a_\theta(e)$ maps a public operation feature e to coordinates in a supplied affine polygon family. For an unadmitted operation, a finite update $\theta \mapsto \theta^+$ may place the two regular endpoint snapshots in different chambers. The parameter interpolation between them may pass through a singular loop, but that interpolation is not executed. Only the separately checked regular snapshots are run. There is consequently no claim that a nonsingular based homotopy can change monodromy. The covering, basepoint, critical set and sheet labels remain fixed. Moreover, a change of homotopy class in the kernel of ρ_D would not count as a new executable operation.

2.3 Terminal boundary feedback and neural credit

An exact endpoint-semantic loss is locally constant in regular loop coordinates. Ordinary within-chamber differentiation alone therefore provides no direct derivative for acquiring a different endpoint action. Let V be raw loop coordinates drawn from a smooth location kernel centered at a_θ, and let $\ell(V;\theta)$ denote the terminal loss obtained by executing that loop and the current neural bindings. Define

$$J_\sigma(\theta) = \int \ell(v;\theta) p_\sigma(v - a_\theta)\, dv. \tag{12}$$

For the current fixed-cover, endpoint-semantic objective, the coupled derivative has two contributions:

$$\nabla_\theta J_\sigma = \mathbb{E}[\partial_\theta \ell(V;\theta)|_{V \text{ fixed}}] + (D_\theta a_\theta)^\top g_{\text{boundary}}. \tag{13}$$

The first term trains the input encoding, readout, calls and their shared backbone while holding the actually observed endpoint maps fixed. For a coordinate slice with crossing events b_r and terminal-loss jumps $\Delta\ell_r = \ell(b_r^+;\theta) - \ell(b_r^-;\theta)$, the second term uses

$$(g_{\text{boundary}})_j = \mathbb{E}_{V_{-j}}\left[\sum_r \Delta\ell_r\, p_\sigma(b_r - a_{\theta,j})\right]. \tag{14}$$

Safe representatives of the neighboring chambers are actually executed to obtain these terminal losses. The implementation uses a normalized raised-cosine kernel and charges event detection, representatives, terminal callbacks and continuation. A coordinate moving one free vertex affects its two adjacent segments; tied coordinates moving several vertices are charged for every affected segment instead of silently inheriting the smaller bound.

Equations (13)–(14) are applications of known differentiation of piecewise-smooth objectives, not a claimed new calculus theorem. A path-work objective with smooth within-chamber dependence on v would require its corresponding continuous derivative as well. The implemented boundary term changes how probability mass is assigned to executable classes; it does not replace successful discrete execution by a mixture of several operations.

The ideal smoothing distribution assigns mass arbitrarily close to a discriminant. The numerical executor additionally requires positive finite clearance. These are distinct domains. The initial score-function arm was censored by a probe entering the exclusion band, rather than assigned a fabricated failure loss. The later hybrid interface evaluates an ideal sample's class at a paid safe representative when available, while still requiring the deterministic candidate loop itself to be executable. This

scalar-family convention is exposed to the conventional controls and is not assumed to solve the general high-dimensional feasibility problem.

2.4 A sufficient protected transaction

For an admitted polygonal loop γ, let $\delta = \min_{t,c\in\mathcal{C}_D} |\gamma(t) - c| > 0$. If its updated vertices satisfy

$$\max_k |v_k(\theta^+, e) - v_k(\theta, e)| < \delta, \tag{15}$$

with the same segmentation and fixed endpoints, linear interpolation gives a based homotopy avoiding $\mathcal{C}_D$. Indeed, the displacement of every point on every interpolated segment is bounded by the maximum vertex displacement; its distance from the critical set remains positive. Hence its endpoint permutation is unchanged. Preserving each admitted generator then preserves every word of those generators under fixed input/output bindings.

This sufficient topological certificate is accompanied by separate checks of the executor's coordinate box and numerical clearance. In the diagnostic the minimum allowed control clearance is .015. Protection of a permutation alone does not protect a changing encoder, router or decoder; the additional binding and soft-execution contracts are specified in Section 3. None of these conditions implies hardware noise protection or a quantum energy advantage from a CPU simulation.

2.5 Executed class acquisition and the unresolved advantage

The initial scalar diagnostic uses two real shared weights,

$$a_{\text{old}} = w_0, \qquad a_{\text{new}} = w_0 + w_1, \qquad (w_0, w_1) = (-3, 4). \tag{16}$$

The old operation B is supplied as the starting mechanism and verified by continuation; it is not credited as learned in an earlier stage. Four terminal training cases require A, which is outside the admitted subgroup $\{I, B\}$. Two accepted neural updates produce

snapshot	a_{old}	a_{new}	training error	old error
initial	-3	1	.75	0
first	-2.625	1.75	.75	0
second	-2.26427259	2.47145482	0	0

The old loop actually changes geometry while retaining B; the new loop crosses its class boundary and acquires A. Four frozen held-out composition cases and an actual checkpoint reload also have zero error. These are finite length-at-most-three tests, not a scaling or long-term continual-learning result. The raw execution is `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/runs/parameter-loop-boundary-learning-v1/learn/result.json`.

The equally informed ordinary event optimizer produces the same weights, parameter hashes and counted trajectory. Discrete chamber search succeeds with one update, 35 program executions and 37 terminal callbacks, versus 70 executions and 72 callbacks for the event arm. Those totals include their disclosed calibration and audit work. A reexecution guard spends more continuation work than the analytic certificate on this instance, but the ordinary optimizer is also allowed that certificate. Thus the result closes the earlier map-first learning gap, while the principal question remains open: does monodromy organization improve acquisition, retention, verification or parameter growth beyond the strongest ordinary constraint and search methods under matched information and complete cost?

3 A trainable loop-based Prometheus Block

The purpose of a Prometheus Block is to join operation formation with executable state updates, learned calls, composition and retention in one trainable state. The present implementation is a concrete fixed-three-state prototype of that interface. It removes the requirement to learn a permutation matrix first and render a loop afterward, but has not yet become a general computational architecture. We specify its actual computation and its observed failure before describing the remaining interface requirements.

3.1 Neural state and computation

The current `HybridLoopBlock` stores a trainable input encoder, a trainable readout, call heads and a shared loop controller. A public feature e is transformed by a shared matrix H; a loop head r produces

$$h_\theta(e) = He, \qquad a_\theta(e) = r^\top h_\theta(e), \qquad \gamma_{a_\theta(e)} : 0 \to a_\theta(e) + i \to a_\theta(e) - i \to 0. \tag{17}$$

The two operation features and two token features are the disclosed vectors $(1, 0)$ and $(1, 1)$. The same H participates in both loop generation and calling, so changing a new operation can change the old operation's geometry and the call probabilities. For call slot s and input token b,

$$p_{\theta,s}(k \mid b) = \mathrm{softmax}(C_s h_\theta(e_b))_k, \qquad k \in \{\mathrm{skip}, 0, 1\}. \tag{18}$$

Skip supplies an identity computation. The other two choices refer to loop bank entries. There is no trainable permutation-score tensor W_k in this prototype: endpoint maps are derived from paid actual root continuation.

Let E_θ and R_θ be row-stochastic matrices obtained from the encoder and readout logits. For an input x and a token word $b_1 \cdots b_T$, the soft learning path is

$$q_0 = E_\theta[x, :], \tag{19}$$

$$q_{t,s+1} = q_{t,s} \left[\sum_k p_{\theta,s}(k \mid b_t) M_k(V) \right], \qquad M_{\mathrm{skip}} = I, \tag{20}$$

$$p_\theta(y \mid x, b_{1:T}, V) = [q_{T,S} R_\theta]_y, \tag{21}$$

where $M_k(V)$ is the one-hot matrix of the actually observed endpoint permutation for the sampled loop coordinates. These matrices are held constant during the within-chamber derivative in Equation (13). The terminal training loss is $-\log p_\theta(y)$, with a declared numerical probability floor of 10^{-12}. The target supplies only the final task output, not a correct route or an intermediate state.

Native execution uses the encoder, call and readout argmax choices and composes the resulting permutations on the same sheet. This is the deployed discrete program against which task success is checked. Cached maps are derived observations of actual loops, not separately learned operations. Reusing a receipt implements the same endpoint semantics without claiming that the changed continuous trajectory has been freshly integrated. Cache namespace, payload, root observations and cold-reexecution audits are recorded and made available to conventional controls.

The executed one-slot model has 30 trainable scalars: 9 encoder logits, 9 readout logits, 4 shared-backbone weights, 2 loop-head weights and 6 call-head weights. It also stores 8 fixed feature-buffer scalars. Both the old operation and its call are supplied initial conditions, while the new call begins uniformly across all three choices. Encoder/readout logits begin near the identity but remain trainable. This starting ability is disclosed rather than counted as previously acquired learning. The exact executed source is `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/versions/code-parameter-hybrid-loop-learning-v1/hybrid_loop_block.py`.

3.2 One learning transaction, three retention obligations

The persistent state comprises neural parameters θ, admitted loop receipts, a reference native binding relation, bounded derived-map storage and the numerical execution contract. Given current terminal examples, a learning transaction proceeds as follows.

1. Sample the unadmitted operation's loop coordinates. Execute or reuse legitimately certified maps, charge their observations, and evaluate the actual soft terminal computation.

2. Compute within-chamber gradients for encoding, readout and calls, together with the class-boundary term. Both derivatives pass through the same neural backbone; no correct new call is inserted manually.

3. Propose an actual shared-weight update. Backtrack its finite scale while checking numerical executability, admitted loop protection, old native binding closure and soft/native agreement. Rejected executions and checks remain in the cost ledger. An aborted transaction restores the original parameter values in place.

4. Admit a new operation and its token binding only after deterministic mean-loop execution gives perfect native and soft accuracy on the stated training admission set and satisfies the expanded confidence contract. The reference then records the actually acquired program, rather than a teacher operation table. Generalization remains a separate held-out test.

These checks discharge different obligations. First, geometric clearance protects the endpoint action of an old loop, as in Equation (15). Second, an old task may still be forgotten if its encoder or call changes, even when every bank permutation is unchanged. The implementation therefore computes a finite relation $\mathcal{R}$ between the reference and proposed native states, starting from all old encoded input pairs, and requires

$$\begin{aligned} &(E^-(x), E^+(x)) \in \mathcal{R}, \\ &(s,t) \in \mathcal{R} \implies R^-(s) = R^+(t), \quad (T_b^-(s), T_b^+(t)) \in \mathcal{R} \quad (b \in \mathcal{A}_{\text{old}}). \end{aligned} \tag{22}$$

Here $E^{\pm}, R^{\pm}, T_b^{\pm}$ denote the complete hard native encoder, readout and token transition before and after the update, including all call slots. Exhausting this finite product-state closure preserves the decoded behavior of every finite old word. It does not require the two internal representations to use identical state labels. This is a conventional finite behavioral-relation check; its correctness is not a topology-exclusive claim.

Third, native preservation alone is insufficient for the soft neural path. Let $\underline{p}_E$, $\underline{p}_R$ and $\underline{p}_C$ be the minimum probabilities assigned to the model's own hard encoder, readout and old-token call choices. Because every executed bank map here is exactly one-hot, the probability of following the entire own-native path through a word of length at most L is bounded below by

$$m_L = \underline{p}_E \, \underline{p}_R \, \underline{p}_C^{SL}. \tag{23}$$

If $m_L > 1/2$, the soft output has the same unique argmax as that native program: the designated path alone carries more than half of the total probability. The current transaction uses a small additional tolerance above $1/2$ and $L = 16$. The mathematical argument is real-arithmetic; the reported float64 evaluation is not interval certification. Native closure covers all finite old words, whereas this particular soft bound covers the specified finite length. Keeping these scopes distinct prevents a finite confidence check from becoming an unsupported unlimited-length neural guarantee.

3.3 Composition and a constructive module interface

The executable interface is a state transformation plus a receipt:

$$(s, b, \theta, \mathcal{M}, \mathcal{K}) \longmapsto (s', y, \text{receipt}), \qquad s' = \Pi_{k_S} \circ \cdots \circ \Pi_{k_1}(s), \tag{24}$$

where $\mathcal{M}$ is the admitted mechanism state and $\mathcal{K}$ contains the execution and protection contracts. Consecutive calls reuse the output sheet, making AB, BA and longer words genuine computations rather than independent classifications. Learning can revise the shared controller and the call structure; admission converts a successfully acquired computation into a protected reusable entry. Stored receipts permit certified semantic reuse, with their maintenance and storage charged separately from parameters.

This interface also identifies the obligations for a general Block. A layer must expose which state representation it passes to the next layer, and learning must preserve the composed input–operation–readout binding, not just each isolated loop. A capacity change must provide a state embedding, transferred operation semantics and an updated protection relation. An irreversible operation would require a disclosed reversible extension or a different state update, charging the extra workspace, cleanup and readout. Continuous inputs require a learned encoding and an explicit domain on which its binding is protected. These are implementable interface requirements; they are not achieved merely by calling the present $D = 3$ module a general Block. The prototype currently supplies fixed finite input features and a fixed covering, and has not implemented capacity growth, noninvertible computation, continuous-domain generalization or verified multilayer stacking.

3.4 Executed hybrid V1: operation formation without task acquisition

Hybrid V1 is an actual negative architecture-development result. The frozen experiment uses four terminal training cases, four held-out composition cases, a supplied old operation, and 100 parameter transactions per arm. The new call is initially uniform; success cannot be obtained by receiving a correct route. The three arms apply the coupled gradient, only its within-chamber component, or only its boundary component. Both components are nevertheless computed and charged in all arms, so their run times are not an ablation efficiency comparison.

arm	transactions	zero displacements	new map	TRAIN soft/native	held-out soft/native
coupled	100	72	A	.25/.25	0/0
within only	100	0	I	.25/.25	0/0
boundary only	100	94	A	.25/.25	0/0

All arms retain old-task soft and native accuracy 1, and none admits the new operation–binding pair. Acquiring the endpoint map A therefore does not establish acquisition of the task that should call it. A "transaction" is explicitly a committed proposal, including a zero-scale proposal; it must not be reported as 100 nonzero learning updates. Raw curves and actual neural checkpoint reloads are preserved in `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/runs/parameter-hybrid-loop-learning-v1/learn/result.json`.

The coupled arm exposes a specific repair target. At its final snapshot the within-chamber call-head gradient norm is approximately 1.2583, but every tested positive joint step scale, down to 1/64, is rejected by the old-loop geometry/map guard; the transaction commits scale zero. Thus learning credit exists while this single-direction transaction cannot use it. The observation does not prove that every feasible update is blocked. It motivates testing a component-wise or constrained

repair that can preserve an admitted class without discarding usable binding credit. Such a fallback is development work, not an executed V1 result, and no outcome is predicted here.

The implementation accounts for terminal callbacks, root continuation, event geometry, neural vector–Jacobian products, proposals and rejections, finite relation checks, cache bytes and real checkpoint storage. Ordinary controls receive the same mathematical certificates and execution observations. Consequently neither the small parameter count nor old-task retention in this failed acquisition test is evidence of an independent Monodromy advantage. The concrete contribution at this stage is the unified trainable interface and an observed, localized failure of its protection–plasticity transaction, with the general-Block and competitive-benefit questions still open.

4 TODO 4: Transfer to Real-World Continual Learning

The same core computation and learning rule must be evaluated on algorithmic, audio, and visual tasks. Input adapters and output heads may change, but changes to the core block or its protection rule define an architecture variant and must be reported separately. Synthetic permutation accuracy alone does not establish this transfer.

4.1 Fixed audio protocol

Speech Commands v2 uses all 35 word classes in seven class-incremental stages, with five new classes at each stage. Training, validation, and test examples follow the official split and all methods use the same 16 kHz input preprocessing. Each stage trains only on its permitted current data and declared memory. Evaluation recognizes all classes seen so far without a task or class-group identifier at inference.

4.2 Fixed visual protocol

Split CIFAR-100 uses ten class-incremental stages, each introducing ten fine-grained classes. The official 50,000-example training set supplies a per-class 10% validation subset under a fixed seed; the official 10,000-example test set is not used for development. All methods share preprocessing and augmentation rules and use a single expanding classifier without a task identifier at inference.

4.3 Memory, initialization, and cost

Both tasks require Memory-0 and Memory-500 comparisons. The latter permits at most 500 old examples across all classes. Saved features, labels, teacher outputs, statistics, and other persistent information are disclosed and charged separately; a representation cache is not free memory. Main experiments use random initialization. Pretrained runs, if any, form a separate comparison with matched pretraining access. The fixed per-stage epoch budgets are 10, 50, and 200, with validation-only configuration selection under the current benchmarking protocol.

4.4 Evidence boundary

No audio or visual training result is available in the present checkpoint. In particular, the existing synthetic learner's public task embeddings cannot be passed to these class-incremental evaluations as inference-time task identifiers. The transfer study must test whether the learned computation actually reduces acquisition, retention, parameter, or full lifecycle costs under these protocols.

Layer	Required scientific coverage	Current evidence boundary
I	Six-stage acquisition at $N = 8, 16, 32$	Boundary-consistent streams at the scales and seeds in Table 1; five-seed protocol incomplete.
II	Noncommuting programs, unseen combinations, lengths 16/32/64, dynamic write/overwrite/read/delay memory	Finite held-out words and length 16 tested; the complete complex-program and dynamic-memory suite is not completed.
III	One Block for reversible, nonreversible, continuous, and variable-length computation	Current finite reversible operations do not establish this generality.
IV	Speech Commands v2 and Split CIFAR-100	No competitive real-data result is claimed in this manuscript checkpoint.
V	Strong applicable architectures, continual-learning controls, and mechanism ablations	Working ordinary program controls; initial neural baselines require further bounded development and validation.

Table 2: All five layers require direct measurements. Evidence at one layer does not replace another layer.

5 Benchmarking and competitive evaluation

Evidence status. The present results are exploratory architecture-development measurements. They do not yet constitute the publication-grade competitive evaluation defined below. In particular, a Transformer that has not acquired the first task cannot serve as decisive evidence of a parameter-efficiency advantage. All recorded failures, configuration trials, retries, maintenance, and extra training remain part of the evidence and cost ledger. A low result is not itself a software defect, and experiments are not extended until a desired number appears.

5.1 Five-layer evaluation scope

Layer II will distinguish operation-order dependence, including $AB \neq BA$ and commutators, from memorization of an observed word distribution. Programs are trained at length at most eight and evaluated separately at lengths 16, 32, and 64. A separate key–value stream requires writing, overwriting, reading, and delayed queries. Every architecture receives terminal task feedback without teacher intermediate execution traces. Layer III keeps the Block's core state-transition and learning mechanism fixed across task types; a change to that core is reported as an architecture variant, rather than silently counted as generalization.

5.2 Information, development budgets, and final evaluation

The synthetic comparison uses the same terminal data, stage order, public context, and evaluation protocol. Dedicated constructors, teacher operator tables, hidden state labels, and correct routes are not free priors for any method. Formation passes and terminal-label accesses beyond minibatch training are charged explicitly. Equal gradient-update counts do not imply equal total learning compute when one method additionally solves a full observation table.

The locked synthetic update limits are B1: 600, B2: 2,400, and B3: 9,600 gradient updates per stage, available to every method including Prometheus. Each main model family has at most 24 development configurations: development seed 0 selects candidates, the top three are checked with development seed 1, and the ranking uses a frozen validation criterion. After configuration lock, the final evaluation uses seeds $0, 1, 2, 3, 4$ without test-based tuning. Shared early checkpoints and

resumed run fragments count as one lineage, not multiple independent seeds. The real-data budgets are 10, 50, and 200 epochs per phase under the same validation-only selection principle.

Only a localized, reproducible software, data, or training-logic error qualifies as a bug fix. Optimizer, architecture, curriculum, or initialization changes are configuration development unless they repair such a demonstrated error. Any supplemental trajectory outside the registered main budget is disclosed separately. Reaching the highest budget without acquisition remains an allowed negative result. Test and OOD measurements do not select configurations, training duration, or stopping points.

Three comparisons are kept distinct: identical information/protocol, matched training and total-compute budgets, and equal attained capability quality. The final analysis requires accuracy–parameter, accuracy–training-cost, and accuracy–inference-cost frontiers. Different method families may use the optimizers and schedules appropriate to their original algorithms; forcing one recipe on all competitors is not a fairness requirement.

5.3 Fixed real-data protocols and strong competitors

Speech Commands v2 uses 35 word classes in seven class-incremental phases of five classes. It uses the official train/validation/test split and common 16-kHz preprocessing. Each model classifies all seen words at inference without a phase identifier or true class group. Split CIFAR-100 uses ten phases of ten fine classes, the official 50,000 training and 10,000 test images, and a fixed class-stratified 10% training holdout for validation. It uses an expanding single head and supplies no task ID at inference. Input processing, augmentation, and data access are matched across architectures.

Both datasets have Memory-0 and Memory-500 conditions, where 500 is the total number of stored old examples across all classes. Stored features, teacher outputs, task statistics, and other historical state are additionally recorded; they are not an exemption from the memory comparison. Main runs start from random initialization. Pretrained variants are supplemental and give every method the same pretraining information. These no-ID class-incremental protocols are distinct from the public-context synthetic benchmark.

The primary architecture family is the Transformer: vanilla, replay, adapters, and replay plus adapters, at two or more demonstrably trainable scales. Parameter-matched and quality-matched comparisons are both required. Continual-learning comparisons include DEN, PackNet, replay, and ER-ACE or an equally mature class-incremental method. PackNet task-incremental results, when applicable, are explicitly separated from no-ID class-incremental rankings. Mamba-2 participates in appropriate sequence/memory tasks, and Titans in tasks suited to its memory interface. HOPE/Nested Learning participates only with a verifiable, applicable implementation. An unfaithful or unvalidated reconstruction is marked `NOT_VALIDATED`; its low accuracy does not constitute a strong-family comparison.

5.4 Measured baseline results at the historical B1 budget

All four runs in Table 3 have zero tasks passing the dual-interface capability criterion. Their low first-task performance shows that the result is not explained by forgetting alone. The $N = 16$ width-64 Transformer similarly has no passing task in the measured stream. These runs cannot establish a competitive architecture advantage for Prometheus.

A separately recorded anchored Transformer trial uses a 100-update source prefix and 2,200 additional updates, totaling 2,300 updates. It is a supplemental configuration trajectory, not a software bug fix or an equal-600-budget result. The fixed development progression stops at its prospective budget. The validation-selected frozen checkpoint is then tested once, obtaining IID 0.37109375,

Method	Total P	First task IID	First task OOD	First task L16	New task at phase 6 IID	New task at phase 6 OOD	New task at phase 6 L16
Transformer-64	103752	.1836	.1230	.1348	.1152	.1094	.1016
Transformer-64+Replay	103752	.1836	.1230	.1348	.1191	.1172	.1055
Transformer-96	229352	.1348	.1094	.1035	.1328	.1406	.1191
Transformer-96+Replay	229352	.1348	.1094	.1035	.1133	.1289	.1152

Table 3: Actual $N = 8$, seed-0 historical streams, each with 600 updates per phase and 3,600 updates overall. "Phase 6" is the newly introduced task at the final phase, not an average over old tasks. These low-acquisition runs are preserved development evidence, not validated strong-Transformer performance.

Policy	Weight bytes	Extra-state JSON bytes	Optimizer tensor bytes	Checkpoint bytes	Old guard seconds
Topology compact	21256	208326	42544	341373	47.4276
Ordinary constraints	21240	104105	42508	202737	45.6787
Frozen + readout	25536	83598	7568	146821	0
Private modules	34656	81825	12316	160265	0

Table 4: Recorded final-phase $N = 8$, seed-0 resource fields for the boundary-consistent policies. The guard covers the phase's 600 updates. These are format-specific fields and overlapping cost scopes, not a disjoint sum or a controlled wall-time ranking.

OOD 0.134765625, and length-16 0.111328125. This improves some short-word behavior without establishing full-task competence; the low test result is not a trigger for unbounded additional trials.

The first 600-update task runs of the adapted DEN-RNN and linear self-modifying CMS reconstruction obtain IID 0.115234375 and 0.138671875, respectively. Their parameter counts are 1,800 and 81,224. The former adapts DEN-style selection, expansion, splitting, and timestamps to a recurrent terminal-feedback interface; the latter is not an official HOPE reproduction. Neither is a validated, tuned strong-family result. They remain `NOT_VALIDATED` for publication-level comparative claims.

For the finite-program question, competent conventional controls are available. At $N = 8$, seed 0, ordinary constrained compact learning matches all six stage-boundary capabilities using 5,310 parameters, versus 5,314 for the topology model. Repaired freezing uses 6,384 and separate task modules 8,664. The frozen repair adds a compatible 128-parameter readout when terminal-profile output multiplicities require it, and later tasks reuse that readout. Its initial failed version remains recorded. These controls explain the present shared/compressed parameter benefit without attributing it to monodromy; they complement rather than replace the required Transformer comparison.

5.5 Costs and causal attribution

All stored and frozen parameters count. Reference arrays are contained within extra-state metadata; cache maps and certificates can also share storage. Checkpoint sizes include optimizer/RNG/container state. JSON size is not minimal sufficient information or peak RAM. Phase wall time includes nested training, evaluation, and protection scopes, which are not added twice. Some jobs overlap on the GPU, so their individual wall times do not support a throughput ranking. One ordinary-control initial-stage report is missing post-training timing fields after a reload-helper failure; these remain null, rather than being reconstructed as measured values.

Complete comparison additionally requires development and failed-run costs, primitive discovery, data accesses, solver and installation work, temporary dense tensors, replay, cache preparation, old-binding checks, rollback, and checkpoint reload. Controlled same-hardware training FLOPs/GPU time, end-to-end inference p50/p95, throughput, and peak memory are not yet complete. Neither software simulation nor these parameter counts measure hardware energy or establish a quantum advantage.

The causal matrix removes, separately, primitive reuse, sharing, workspace growth, old-capability protection, graph constraints, and native compilation. Topology-specific comparisons substitute ordinary constraints for monodromy, disable cross-class loop learning, disable within-class safe deformations, and compare ordinary cross-class optimization and discrete search under the same information and costs. Shared compression, program formation, graph closure, and generic consistency repair are not credited to monodromy. A same-mechanism reimplementation tests executable correspondence; it is not by itself a historical-priority finding or a general impossibility result.

5.6 Statistical reporting and completion status

Formal comparisons use all five independent final seeds, mean, standard deviation, and 95% confidence intervals; paired-seed differences and their intervals are primary. Multiple primary tests receive multiplicity correction, and failed seeds remain included with their actual censoring or failure status. The principal outcomes are final/average-incremental accuracy, forgetting, backward transfer, acquisition cost, and total parameters and resources at equal retained quality. Current three-seed growth observations do not satisfy that five-seed protocol and are not promoted to formal significance results.

Publication-grade benchmarking remains `INCOMPLETE`: strong trained neural comparators, every empirical layer, the two fixed real-data benchmarks, complete causal attribution, and full resource/statistical evidence are still required. No amount of success on the current finite task substitutes for these requirements.

Evidence locations. Original neural streams and their actual checkpoints reside under `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/runs/`, including `parameter-n8-seed0-transformer64-full`, `parameter-n8-seed0-transformer64-anchored-competence-v1`, `parameter-n8-seed0-den32-through0-v2`, and `parameter-n8-seed0-selfmod_cms64-l1-through0-v2`. The frozen stage-level extract is `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/parameter-boundary-growth-evidence.csv`. Actual original-source reading and adaptation boundaries for DEN are in `.deep-work/2026-10-31T234403-0400-v2/sources/parameter-den/reading-and-adaptation.md`. The governing prospective benchmark protocol is `ALL.md`, TODO 5.

6 Theory, evidence boundaries, and reproducibility

The role of theory is to identify the conditions under which an executable operation, its task binding, and its soft neural realization remain correct. These are distinct obligations. A mathematical invariant is not a proof of task acquisition, a finite successful training stream is not a generalization theorem, and an independent agent's agreement is not a scientific certificate. The results below describe conditional finite mechanisms; they do not assert an original general learning principle or a topology-specific cost advantage.

6.1 Continuation witnesses and finite-state identification

Proposition 1 (Observed state separation)**.** *Fix a deterministic recurrent system and one task context. Suppose histories h and h' have different observed outputs under the same continuation s. Then any deterministic internal-state realization of both observations must assign different recurrent states to h and h'.*

Proof. If their recurrent states were equal, applying the same continuation under the same transition rule would produce equal final states and decoded outputs, contrary to the observations. □

Thus m pairwise distinct terminal continuation profiles require at least m distinguishable states in this deterministic representation. This justifies the current witness-triggered workspace expansion. It does not show that equal finite profiles are equivalent under all unobserved suffixes. The implemented closure and full-TRAIN checks identify a model consistent with available observations; its held-out behavior remains an empirical question. No exponential global-closure conclusion is inferred from this finite witness.

6.2 Binding-level protection under state expansion

Let an old task model be (X, ι, T, r) and an updated model be (X', ι', T', r'), with a shared observable input domain and alphabet $\mathcal{A}$. The encoders are ι, ι', transitions are T_a, T'_a, and readouts are r, r'. A relation $\mathcal{R} \subseteq X \times X'$ protects the old computation if

$$(\iota(x), \iota'(x)) \in \mathcal{R} \quad \text{for every protected input } x, \tag{25}$$

$$(z, z') \in \mathcal{R} \implies r(z) = r'(z'), \tag{26}$$

$$(z, z') \in \mathcal{R} \implies (T_a(z), T'_a(z')) \in \mathcal{R} \quad (a \in \mathcal{A}). \tag{27}$$

Proposition 2 (Native execution preservation)**.** *Under these conditions, the old and updated systems have the same decoded output for every finite word over the protected alphabet, starting from any protected input.*

Proof. The empty-word state pair belongs to $\mathcal{R}$ by (25). Each next token preserves membership by (27). Induction on word length and readout agreement give the result. □

The relation can connect different-sized state spaces, so it permits expansion and internal reorganization without requiring every old parameter to freeze. It protects equivalence to the admitted reference, not correctness of that reference against an unknown task. Checking a finite reachable relation is ordinary semantic verification, not by itself a topological contribution. The reference construction, breadth-first exploration, acceptance/backtracking, and stored relation all have measured maintenance costs.

6.3 What continuous-loop protection establishes

For the present permutation-valued realization, a loop in the unordered configuration space of D distinct points induces a permutation of the marked fiber. With a fixed initial labeling, monodromy gives a representation

$$\rho : \pi_1(\mathrm{Conf}_D(\mathbb{C})/S_D) = B_D \longrightarrow S_D. \tag{28}$$

The quotient action records endpoint permutations; it is not a faithful representation of the whole braid group. Different braid classes, including pure braids, can have identical permutation actions. Ordinary graph equivalence is likewise not a new topological invariant merely because it is used alongside this realization.

Proposition 3 (A clearance condition for based-loop retention)**.** *Let $X = \mathbb{C} \setminus \mathcal{B}$ for a fixed finite branch set $\mathcal{B}$ and a fixed covering over X. Let γ be a polygonal based loop with clearance $\delta = \inf_{s,b\in\mathcal{B}} |\gamma(s)-b| > 0$. Let γ' have the same parameterized polygon structure and the same basepoint, and suppose the maximum corresponding vertex displacement is less than δ. Then γ and γ' are based-homotopic in X and induce the same marked-fiber monodromy.*

Proof. Linear interpolation on each segment bounds every pointwise loop displacement by the largest vertex displacement. For $H(t,s) = (1-t)\gamma(s) + t\gamma'(s)$, $|H(t,s) - b| \geq \delta - \sup_s |\gamma'(s) - \gamma(s)| > 0$. Thus the interpolation misses every branch point and fixes the endpoints. Homotopy invariance of covering-space lifting gives the same monodromy. □

This is a conditional geometric protection rule: it permits real continuous motion, rather than manually freezing all loop parameters. The covering, singular set, basepoint, and labeling must satisfy the stated fixed conditions; changing them requires its own argument. Applying it to a learned family also requires a bound for every protected loop/input in the claimed domain. Numerical continuation, finite-grid checks, and floating-point clearances are not automatically rigorous interval certificates. A certified deformation family may reuse an already paid-for realization across parameter changes, but its assumptions, initial lift, invalidation, and checkpoint reconstruction must all be included in the resource account.

Loop-class preservation, task-binding preservation, and accurate finite-time execution remain separate. A gate can remain topologically unchanged while a shared task router selects the wrong gate. Conversely, unchanged endpoint behavior under a conventional constrained program is not evidence that its learning benefit is topological. The software monodromy simulator is not a physical quantum computer; no hardware energy or quantum speedup follows from its numerical execution.

6.4 A sufficient soft/native consistency bound

Consider nonnegative stochastic soft execution as in (3). Suppose the hard computation uses at most $SL_{\max}$ operation slots. Let p_E, p_R, p_C, p_O be lower bounds on the probabilities assigned, respectively, to the selected native encoding, selected readout, every selected slot call, and every selected operation edge. The probability mass following that particular native execution path is at least

$$p_{\text{path}} \geq p_E p_R (p_C p_O)^{SL_{\max}}. \tag{29}$$

If this bound exceeds $1/2$, the native output is the unique soft argmax for every covered word, because all alternative output mass together is less than $1/2$. The same conclusion can hold with a weaker bound, since several paths may decode to the same output; this is only a sufficient condition.

The actual refresh evaluates these minima from the model's current soft operation tensors and its own hard calls. It checks parameter count, original slot structure, current native graphs, and frozen alias integrity. Subsequent weight or temperature changes require renewed checking. Floating-point evaluation of the analytical bound is not interval-certified verification. The result transfers current native correctness to a bounded-length soft interface; it does not teach a new class or prove always-online retention between maintenance points. The rule is available to the non-topological controls and therefore cannot establish a monodromy-exclusive effect.

6.5 Executed evidence and version lineage

Every reported run is tied to actual task arrays, configuration, code version, raw update/formation/guard records, curves, saved neural tensors, extra state, and reload checks. Synchronization manifests supply file hashes for completed copies. A successful exit alone does not demonstrate learning; measured

predictions and restored-checkpoint behavior supply the execution evidence. Missing records are retained as missing. Failed code, unsuccessful model versions, and additional computation remain separate from the eventual successful trajectory, rather than being relabeled as if success occurred earlier.

The boundary evidence extraction reads original JSON/JSONL records and safe ZIP metadata without training or executing checkpoint code. Its CSV exposes stage-boundary outcomes and intermediate curves separately. Subsequent additional seeds remain traceable to their own original run manifests rather than being silently attributed to that older extract. Frozen-backbone storage audits additionally check tensor aliasing and unchanged old weights; these audits establish stored values and counts, not independent neural inference. The raw GPU reload records provide the latter within the execution environment.

Reproducibility artifacts. The source, data generators, and raw run tree are under `.deep-work/2026-10-31T234403-0400-v2/probes/topology-architecture/`. The finite formation implementation is `terminal_profile_formation.py`, the compact model is `parameter_native_models.py`, the experiment driver is `parameter_efficiency_study.py`, and neural/native maintenance is `parameter_consistency_refresh.py` within that directory. Independent within-campaign examinations are preserved at `.deep-work/2026-10-31T234403-0400-v2/supervision/reviews/review-037-boundary-growth-and-frozen-readout.md` and `.deep-work/2026-10-31T234403-0400-v2/supervision/reviews/review-038-loop-boundary-learning.md`. The latter concerns a distinct, small terminal-feedback loop-class diagnostic; it does not substitute for the six-stage architecture or publication benchmark.

The frozen growth extract's SHA256 is `311a5073e8f777fc6c6270d5323ebee258542b9be15a50d1bae2fa3ddb3e6c12`, and its learning-curve CSV has SHA256 `b7e9a6b9e11ce327e0e2b1d108ae3ea95b82e3f1483f32cc7e51d50f88af28b4`. These identify particular evidence snapshots, not a claim that every later artifact or current source has the same hash. Each executed version and later run must retain its own provenance.

6.6 Independent checking and limits of present claims

The campaign uses independent agent contexts to inspect original equations, code, raw outputs, costs, and saved tensor metadata. Such inspection has corrected coding, physical-model, cost, and claim-scope errors. It is not independent laboratory replication, community peer review, or proof by model consensus. The finite propositions above are mathematical arguments in the stated models; this manuscript does not claim Lean compilation or formal verification of their implementations. Primary-source reading records preserve what was actually read and the boundaries of adaptations. Historical novelty requires a checked comparison with substantive original work, beyond an independent reimplementation of the same rule.

The final reproducibility package requires fixed data splits, all five final seeds, locked configurations, executable environments, checkpoints, curves, raw failures, causal ablations, and statistical analysis scripts. Formal confidence intervals and paired comparisons remain to be produced under the protocol in Section 5; successful development seeds are not selected as their substitute. Complete cost reporting must include development, discovery, formation, maintenance, storage, verification, and inference in addition to gradient updates.

At this checkpoint, the supported result is a finite, executable continual acquisition mechanism with measured boundary consistency and working ordinary controls. An independently useful topological learning contribution, a general nonreversible/continuous Block, competitive real-data results, and the complete publication-grade evidence package remain unresolved. This manuscript

is a working scientific record, not a declaration of breakthrough, publication readiness, or public release; authorship and intellectual-property conditions must be settled before external dissemination.